

INDEX

Sl. No.	Contents	Page No.
1	ABSTRACT	3
2	INTRODUCTION	5
3	SYSTEM STUDY	7
	3.1. PROBLEM STATEMENT	
	3.2. EXISTING SYSTEM	
	3.3. PROPOSED SYSTEM	
4	MODULE DESCRIPTION	10
	4.1. MODULES	
5	FEASIBILITY STUDY	13
	5.1. OPERATIONAL FEASIBILITY	
	5.2. TECHNICAL FEASIBILITY	
	5.3. ECONOMIC FEASIBILITY	
	5.4. BEHAVIORAL FEASIBILITY	
6	SYSTEM REQUIREMENT SPECIFICATION	17
	6.1. SOFTWARE SPECIFICATION	
	6.2. HARDWARE SPECIFICATION	
7	SYSTEM DESIGN	20
	7.1. INITIAL DESIGN	
	7.2. INPUT DESIGN	
	7.3. OUTPUT DESIGN	
	7.4. DATA FLOW DIAGRAMS	
	7.5. DATABASE DESIGN	

8	SYSTEM DEVELOPMENT	32
	8.1. FRONT END	
	8.2. BACK END	
9	SYSTEM TESTING	38
	9.1. SYSTEM TESTING	
10	SYSTEM IMPLEMENTATION & SYSTEM MAINTAINANCE	42
	10.1. SOFTWARE IMPLEMENTATION	
	10.2. SYSTEM MAINTAINANCE	
11	SAMPLE CODE	47
12	SCREENSHOT	62
13	SCOPE FOR FUTURE ENHANCEMENT	67
14	CONCLUSION	70
15	REFERENCE	72

ABSTRACT

The “Sign Link” is a pioneering web-based application specifically engineered to bridge the profound communication gap between the hearing-impaired community and the general public. While traditional video calling platforms rely almost exclusively on audio-based communication, they often present a significant accessibility barrier for individuals who are deaf or mute. This project strategically eliminates that limitation by integrating a bidirectional, real-time translation engine directly into a specialized video call interface. By leveraging sophisticated computer vision and machine learning algorithms, the system creates a seamless environment where visual signs are instantly converted into readable text for hearing participants, while spoken words are simultaneously captured and displayed as live captions for deaf users.

The application offers a comprehensive user experience that begins with a secure registration and login system, allowing users to build a personal network by sending and accepting friend requests. Once connected, the core functionality is divided into two distinct modes tailored to the specific needs of the users. In the mode designed for deaf and mute individuals, the system tracks hand gestures and skeletal landmarks to translate sign language into text with high precision. Conversely, for the hearing participant, the system utilizes speech-to-text processing to ensure the deaf user can follow the conversation without delay. To complement these features, a dedicated real-time chat option is integrated into the platform, allowing friends to exchange text messages both

during and outside of active calls, which is essential for sharing complex data such as links, addresses, or technical terms that may be difficult to sign or speak.

The technical foundation of the application ensures a robust and responsive experience across various devices. The frontend is developed using a combination of HTML and JavaScript to handle the dynamic requirements of live video streaming and interface reactivity. The backend logic is powered by the Python Django framework, which provides a secure and scalable environment for managing user data and friend relationships. All data is structured within a MySQL database, efficiently managed via SQLyog, and the entire development process is optimized within the PyCharm IDE. By synthesizing these modern technologies, the project effectively demonstrates how innovative software design can be leveraged to foster inclusivity, dismantle social barriers, and ensure that effective communication is a universal right for everyone.

INTRODUCTION

INTRODUCTION

Communication is the fundamental essence of human interaction, yet for millions of individuals who are deaf or mute, standard communication channels often present insurmountable barriers. While video calling technology has connected people globally, it remains an audio-centric medium, fundamentally excluding those who rely on Sign Language as their primary mode of expression.

The Sign language translator via video call project is a dedicated initiative to create a truly inclusive digital environment. It directly addresses the challenge of facilitating real-time communication between a person who utilizes sign language and a person who does not. Historically, this interaction has required a human interpreter—a solution that is often not feasible, immediate, or private.

This project proposes a sophisticated software solution that acts as a continuous digital interpreter. By integrating sign language recognition and speech-to-text technologies into a unified web application, users can communicate without an intermediary. The system identifies signs made by a mute person and instantly converts them into text for the hearing participant. Simultaneously, it captures spoken words and converts them into text for the deaf user, ensuring a fluid dialogue.

To enhance the user experience, the platform includes a social layer where users can connect, add friends, and utilize a real-time chat feature for text-based messaging. Built using a modern tech stack including Python Django and JavaScript, this application serves as a comprehensive social platform. It allows users to initiate video calls that break the silence, making communication accessible, reliable, and universal for everyone.

SYSTEM STUDY

- 3.1. PROBLEM STATEMENT
- 3.2. EXISTING SYSTEM
- 3.3. PROPOSED SYSTEM

3.1. PROBLEM STATEMENT

Current digital communication tools like Zoom and WhatsApp are fundamentally built for the hearing world, leaving the deaf and mute community at a disadvantage. While video is available, it only facilitates a bridge if both parties understand sign language. Without this mutual knowledge, a "communication vacuum" is created: deaf individuals are forced to rely on exhausting lip-reading or inaccurate real-time captions, while hearing users remain unable to interpret the complex gestures and facial expressions that constitute sign language.

Beyond these technical hurdles, there is a significant lack of integrated platforms that combine social networking with accessible video tools. Most existing ecosystems treat "friend discovery" and video communication as separate layers, ignoring the need for a unified space designed for universal access. This highlights the urgent need for a solution that merges a social networking interface with a real-time sign language translation engine, finally allowing signers and non-signers to connect, interact, and build relationships autonomously.

3.2. EXISTING SYSTEM

Currently, the recruitment and management of migrant workers rely on traditional methods like manual paperwork, phone calls, and fragmented digital systems. Contractors manage workers using spreadsheets or isolated software, while workers often find jobs through informal networks. Job providers post openings via local agencies or online boards, but there is no seamless connection with contractors or workers. Communication is typically handled through phone calls or separate messaging apps, leading to inefficiencies and delays.

While some platforms exist for job posting or contractor management, they lack integration and essential features like real-time progress tracking, AI based estimates, and centralized worker profiles, making the current systems inefficient and prone to errors.

3.3. PROPOSED SYSTEM

- **MediaPipe-Powered Translation:** The core translation layer utilizes MediaPipe's Hand Tracking solution directly within the browser. It detects hand landmarks with high fidelity to recognize gestures in real-time, converting Sign Language into Text for the hearing user without requiring heavy server-side processing.

- **High-Quality Video Infrastructure:** The application leverages the Google Video API (or WebRTC) to handle the heavy lifting of video transmission. This ensures stable, low-latency video calls and handles bandwidth optimization, allowing the custom translation layer to focus purely on gesture recognition.

- **User Management:** To foster a social environment, the system includes a secure Authentication Layer for registration and login. It features a dedicated Social Dashboard where users can manage friend requests, maintain a contact list, and initiate calls with verified connections instantly.

- **Edge-AI Performance:** By running MediaPipe on the client side (Edge AI), the system minimizes latency. Gestures are processed instantly on the user's device rather than being sent to a cloud server, ensuring that the Speech-to-Text and Sign-to-Text conversion happens in near real-time, syncing perfectly with the video feed.

- **Web-Based Architecture:** The application is built using a cross-platform web framework, making it accessible via any standard browser. Because MediaPipe is highly optimized for mobile and web, users can access these advanced accessibility features on basic laptops or smartphones without needing expensive GPUs.

MODULE DESCRIPTION

4.1. MODULE DESCRIPTION

1. ADMIN

The Admin Module acts as the centralized command center, providing the necessary tools to oversee platform security and maintain community standards. Administrators possess the high-level authority to view a real-time list of all currently logged-in users, ensuring they can monitor system traffic and platform engagement effectively. A significant portion of this module is dedicated to maintaining a transparent and responsive communication loop between the management and the user base.

Administrators have full access to a specialized dashboard where they can review all user-submitted feedback to identify areas for system improvement. Furthermore, the module includes a comprehensive complaint management system that allows administrators to examine detailed reports sent by users and issue direct, formal replies to resolve grievances. This capability ensures that technical or social issues are addressed promptly, fostering a reliable and inclusive environment for every participant on the platform.

2. USER

The User Module serves as a comprehensive social and accessibility hub, designed to facilitate seamless interaction both online and offline. It begins with a secure authentication framework that manages registration and login, allowing individuals to maintain a personalized profile that they can view and update at any time to reflect their current status. Social connectivity is integrated directly into the platform, enabling users to browse a directory of all registered members and build a network through a friend request system where they can send, accept, or reject connections.

The module's primary innovation is its dual-mode translation capability. For online interactions, it features a robust video calling interface that performs real-time bidirectional translation: interpreting the deaf user's sign language into text while simultaneously converting the hearing participant's voice into text. To further bridge the gap during physical encounters, the module includes a specialized Offline Detection feature. This functions as a standalone Python application that enables live, real-time interaction between two people using a single computer system. It interprets sign language into both text and speech while translating spoken words back into text, ensuring accessibility is maintained even without a remote connection. Finally, users can actively contribute to the platform's growth by submitting formal feedback and complaints through dedicated communication channels.

FEASIBILITY STUDY

- 5.1 OPERATIONAL FEASIBILITY
- 5.2 TECHNICAL FEASIBILITY
- 5.3 ECONOMIC FEASIBILITY
- 5.4 BEHAVIORAL FEASIBILITY

FEASIBILITY STUDY

A feasibility study is a preliminary study undertaken to determine and document a project's viability. The results of this study are used to make a decision whether to proceed with the project. If it indeed leads to a project being approved, it will – before the real work of proposed project starts – be used to ascertain the likelihood of the project's success. It is an analysis of possible alternative solutions to a problem and recommendation on the best alternative. It, for example can decide.

Whether an order processing be carried out by a new system more efficiently than the previous one. The feasibility study proposes one or more conceptual solutions to the problem set for the project. The conceptual solution gives an idea of what the new system will look like. They define what will be done on the computer and what will remain manual. It also indicates what input will be needed by the system and what outputs will be produced. These solutions should be proven feasible and a preferred solution is accepted .

The feasibility study environment enables all alternatives to be discussed and evaluated. This phase starts with an identification of the main characteristics of the required system. During this stage it is important to collect information as much as possible about the software package that might meet the specification from as many sources as possible.

Facts considered in the feasibility analysis were:

- Operational Feasibility
- Technical Feasibility
- Economic Feasibility
- Behavioral Feasibility

The requirements of the system are specified with a set of constraints such as system objectives and the description of the outputs. Three key factors are to be considered during the feasibility study.

5.1. OPERATIONAL FEASIBILITY

The proposed project is said to be beneficial only if they can be modules and carved out in to a system that will meet all the requirement. The best for the feasibility prohibits if the system would run without faults when it is deployed or there are any major hurdles to be the implementations of the proposed system. The proposed system is not supposed to cause any harm to the user or the computer that is being used and the system is safe and secure.

5.2. TECHNICAL FEASIBILITY

The proposed system meets all the requirement of the Technical feasibility. Because of the implementation of the project need no technological difficult. The web part of the project can be easily implemented in a user friendly IDE, that is in Visual Studio Code.

So the development of the project does not meet any type of the technical difficult. Hence we can be say that the proposed system is technically feasible.

5.3. ECONOMIC FEASIBILITY

The proposed project is economically feasible. Because once the system is put into its use in the current market the system provides economical advantage to the firm. Also the firm can afford the cost to implement the project. The proposed project provides tangible and intangible benefits comparing to the existing projects. This system does not need any initial investments and it can improve the quality of service.

5.4 BEHAVIORAL FEASIBILITY

The proposed system has high operational feasibility. Because the site is accessible anywhere and anytime. For operating the application, we only need an internet connection and a smartphone. The changes in the hardware or software environment do not affect the system.

People inherently change the computer or mobile phones. But the changes in the environment do not affect the system.

SYSTEM REQUIREMENT SPECIFICATION

- 6.1. SOFTWARE SPECIFICATION
- 6.2. HARDWARE SPECIFICATION

6. SYSTEM SPECIFICATION

Hardware and software requirements for the installation and smooth functioning of this product could be configured based on the requirements needed by the component of the operating environment that works as a front-end system. Here we suggest minimum configuration for both hardware and software. It includes two phases.

6.1. Software Requirements

6.2. Hardware Requirements

6.1. Software Specification

One of the most difficult task is selecting software for the system, once the system requirements is found out then we have to determine whether a particular software package fits for those system requirements. The application requirement:

- Front end : HTML/JS/CSS
- Back end : MySQL, Python
- Operating system : Windows 7 or above
- IDE : PyCharm, SQLyog

6.2. Hardware Specification

The selection of hardware is very important in the existence and proper working of software. The selection hardware, the capacity and speed are also important.

- Processor : Intel Pentium Core i3 and above
- Primary Memory : 8GB RAM and above
- Storage : 320 GB hard disk and above
- Display : VGA Color Monitor
- Key Board : Windows compatible
- Mouse : Windows compatible

SYSTEM DESIGN

- 7.1. INITIAL DESIGN
- 7.2. INPUT DESIGN
- 7.3. OUTPUT DESIGN
- 7.4. DATA FLOW DIAGRAM
- 7.5. DATABASE DESIGN

7. SYSTEM DESIGN

System design is the process of developing specifications for a candidate system that meet the criteria established in the system analysis. The major step in system design is the preparation of the input forms and the output reports in a form applicable to the user.

The system design is the most creative and challenging phase. The first step is to determine how the output is produced and in what format. Samples of input and output are presented. Next, the input data and the master data are to be designed to meet the requirements of the proposed output. The operational phases are handled through program construction testing, including a list of programs needed to meet the system objective and complete documentation.

7.1. INITIAL DESIGN

The initial design of the app is focused on developing a user-friendly interface that meets user requirements efficiently. For that purpose, we have explored well-known technologies and identified Flutter as the ideal framework for development. Flutter provides a rich set of pre-built widgets that allow for a highly customizable and visually appealing interface while ensuring a consistent user experience across different platforms.

We gathered requirements for this application through various approaches and found that Flutter's widget-based UI framework effectively enables the creation of a responsive and aesthetically pleasing interface. The application is built using the Dart programming language, ensuring efficient performance and seamless integration of functionalities.

7.2. INPUT DESIGN

Input design is the process of converting the user originated inputs to a computer format. The input design involves determining what the inputs are, how the data should be performed, how to validate data, how minimize data entry and how to provide a multiuser facility. The design for handling input specifies how data are accepted for computer processing. Input design is a part of overall system design that needs careful attention and if includes specifying the means by which actions are taken.

A system user interacting through a system must be able to tell the system whether to accept input produce a report or end processing. The collection of input data is considered to be most expensive part of the system design. Since the inputs have to be planned in such a manner so as to get the relevant information extreme care is taken to obtain the information. If the data going into the system is incorrect then processing and outputs will magnify this error. All input data are validated in the order and if any data violates any conditions, the user is warned by a message. If the data satisfies all the conditions, then it is transferred to the appropriate tables in the database.

We have to keep in mind the following things to design the system

- What data to input?
 - What medium to use?
 - The dialogue to guide users in providing input.
 - Methods for performing input validation and steps to follow when errors occur
- Input requirement gathering was one of the major trivial process in web or android application development. The project involves text inputs. The inputs can be entered through keyboard and mouse. The text input is gathered by forms with text boxes.

7.3. OUTPUT DESIGN

Effective output design will improve the clarity and performance of output. Output design phase of the system is concerned with the convergence of information's to the end user friendly manner. The output design should be efficient, intelligible so that system relationship with the end user is improved and thereby enhancing the process of decision making.

Outputs are the most important and direct source of information to the user and to the management. Output design generally deals with the results generated by the system.

7.4. DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of —flow of data through an information system, modelling its process aspects. Often they are preliminary step used to create the over view of a system which can later be elaborated. DFD also be used for visualization of data processing (Structured design).

A DFD shows what kind of information will be input to and output from the system, where the data will come from and go to and where the data will be stored. It does not show information about the timing of process, or information about whether process will operate in sequence or in parallel.

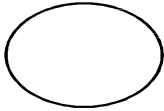
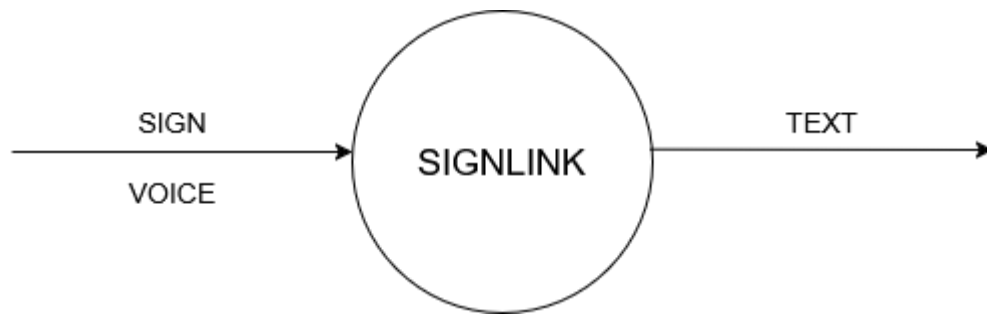
Symbol	Name	Function
	Process	Performs some transformation of input data to yield output data.
	Data Flow	Used to connect processes to each other, to sources or sinks; the arrow head indicates direction of data flow.
	Source or sink(External Entity)	A source of system inputs or sinks of system outputs.
	Data store	A repository of data.place where data is stored.

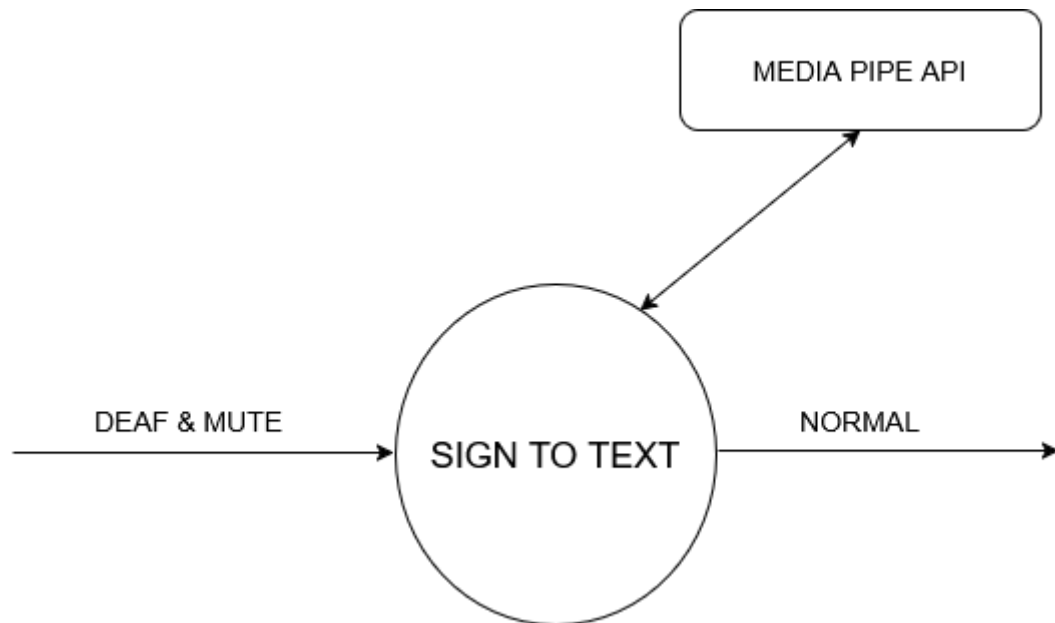
Fig: Above depicted are the major shapes used in DFD.

The DFD at the simplest level is referred to as the context analysis diagram. These are referred to as the explaining its process in detail. Processes are numbered for easy identification and the data stores, source and destination of data are normally labeled in block letters. Each data flow is labeled for easy understanding.

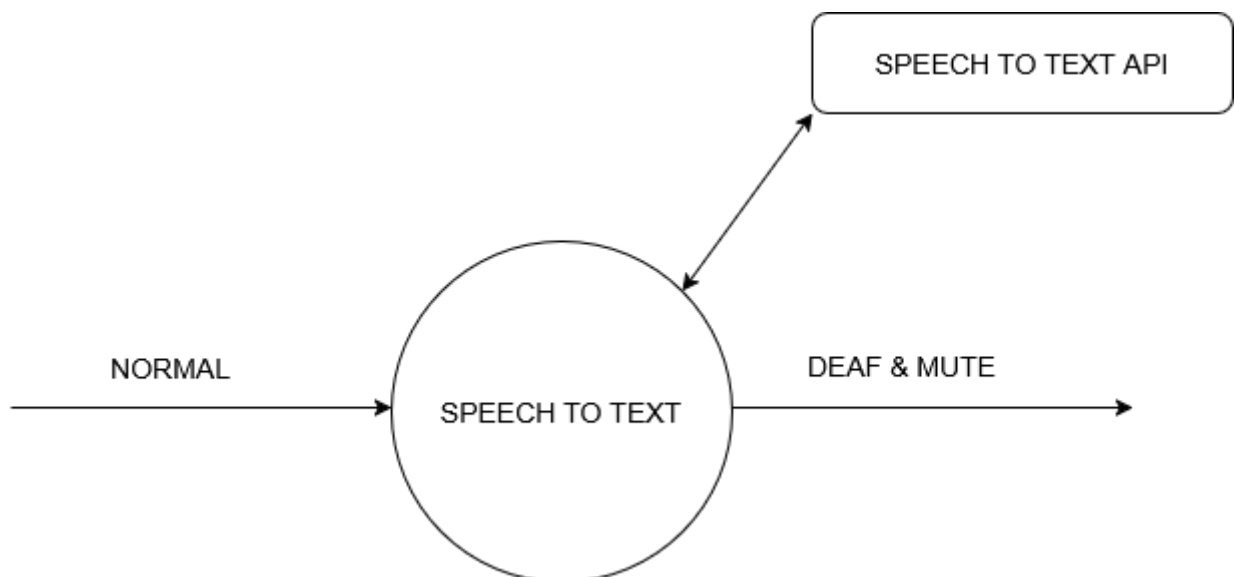
LEVEL 0



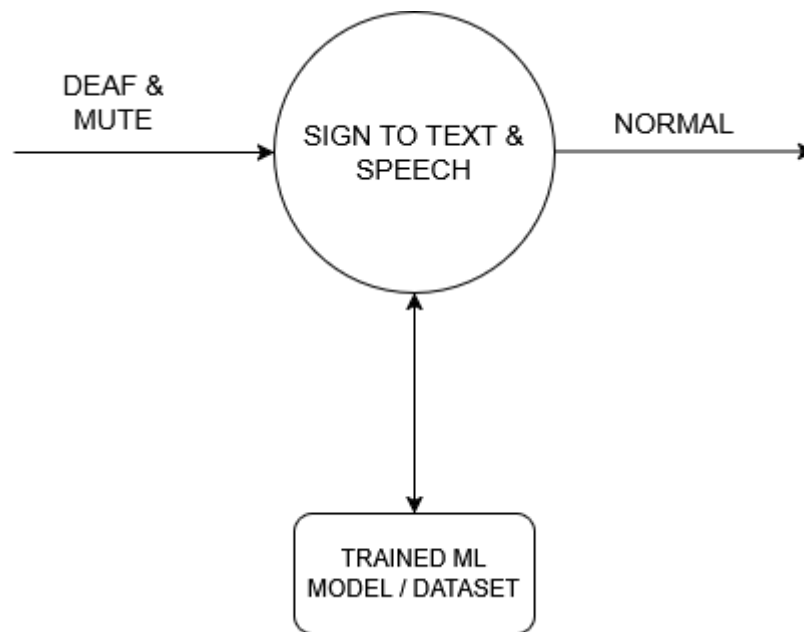
LEVEL 1 - ONLINE



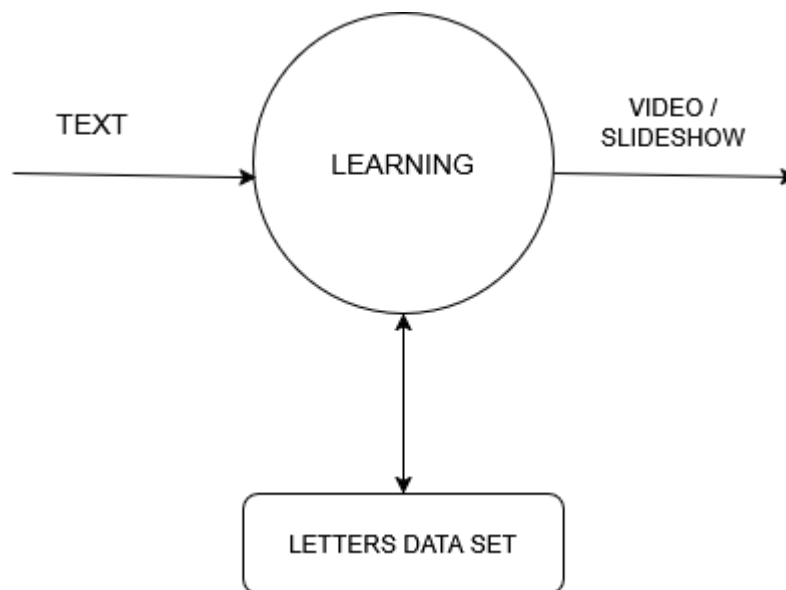
LEVEL 1 - ONLINE



LEVEL 1- OFFLINE



LEVEL 1- LEARNING



7.4. DATABASE DESIGN

The database design is built on a robust relational architecture using MySQL, with SQLyog serving as the graphical interface for efficient schema management and query execution. At the core of the data model is the `auth_user` table, which leverages Django's built-in authentication system to securely handle essential login credentials like usernames, emails, and hashed passwords. To maintain a clean and normalized architecture, extended user information—such as profile pictures and personal details—is stored in a separate `user_profile` table that links directly back to the primary user record. Furthermore, the application's social connectivity is managed through a dedicated friends mapping table. This table maps the network of user interactions by tracking the `sender_id`, `receiver_id`, and the current connection status (such as pending, accepted, or rejected), which serves as the logical foundation for establishing the peer-to-peer relationships required to initiate video calls.

FIRST NORMAL FORM (1NF) Equivalent

A table in MySQL is said to be in First Normal Form (1NF) if it contains only atomic (indivisible) values and eliminates repeating groups or multi-valued attributes. Each column stores a single value, and every record can be uniquely identified using a primary key.

In this project, database tables are structured so that no repeating columns exist, multi-valued attributes are separated into individual rows, and composite attributes are divided into separate fields. First Normal Form principles are applied across all tables to ensure organized and easily retrievable data.

SECOND NORMAL FORM (2NF) Equivalent

A table in MySQL is said to be in Second Normal Form (2NF) if it satisfies all the conditions of 1NF and ensures that non-key attributes are fully dependent on the entire primary key rather than a part of it.

Partial dependency is removed by separating related data into different tables and establishing relationships using foreign keys. This minimizes redundancy and improves data consistency. In this database design, related information is stored in separate tables, ensuring that each table contains only data directly related to its primary key. Second Normal Form structuring is implemented across all tables in the database.

THIRD NORMAL FORM (3NF) Equivalent

A table in MySQL is said to be in Third Normal Form (3NF) if it satisfies all the conditions of 2NF and ensures that non-key attributes do not depend on other non-key attributes.

Transitive dependencies are eliminated by separating indirectly related data into independent tables connected through foreign key relationships. This prevents unnecessary duplication and maintains data integrity. In this project, dependent attributes are stored in appropriate related tables rather than within the same table, ensuring an efficient, scalable, and consistent relational database structure. Third Normal Form principles are applied throughout the database design.

TABLES

- **Table: Users**

id	name	email	phone	photo	LOGIN_id
12	sange	sangee@gmail.com	6278335383	/media/uhdpaper.com-6821e-pc-4k_G49DrI4.jpg	33
13	Vimalraj	raj68vim@gmail.com	9605713626	/media/uhdpaper.com-6021e-phone-4k.jpg	34
14	cinema	cinema@gmail.com	9895789012	/media/2.jpg	36
15	blaa	anuamalleo1@gmail.com	8989898989	/media/87292_2017-29.jpg	38
16	blaa	anuamalleo2@gmail.com	9876543211	/media/87292_2017-29_TqGksWr.jpg	40
17	me	anuamalleo3@gmail.com	9876543211	/media/16907_68898_jj5f555dsmm.jpg	41

- **Table : Complaints**

id	complaint		complaintdate	reply		replydate	USER_id
1	this is not working properly	30B	2025-09-16	it will be fix soon	19B	2025-09-18	1
2	helloio hi	10B	2025-09-26	welcome	7B	2025-09-30	2
3	what the hell brother	21B	2025-12-02	fweffwffewfewfewfew...	24B	2025-12-03	10
4	This site is toooooo good!!!!!!1	33B	2025-12-02	m jjjjjjjjjjjjjjjjjjjj...	28B	2026-01-26	14
5	Improve moree.....	25B	2026-01-26	Update is cominggg	18B	2026-02-03	10
7	there is a lilttle problem in offline d...	47B	2026-02-25	it will be solved soon	22B	2026-02-25	10
(Auto)	(NULL)	OK	(NULL)	(NULL)	OK	(NULL)	(NULL)

- Table : Feedbacks

id	feedback		date	USER_id 
1	anu is playboy	15B	2025-11-07	1
2	amal is amazing	15B	2025-08-28	2
3	this is a really good...	29B	2025-12-02	10
4	very good website	17B	2026-03-18	10
(Auto)	(NULL)	OK	(NULL)	(NULL)

SYSTEM DEVELOPMENT

- 8.1. FRONT END
- 8.2. BACK END

8. SYSTEM DEVELOPMENT

8.1. FRONT END:

HTML (HyperText Markup Language)

HTML acts as the essential "skeleton" of your application. In this project, it goes beyond simple text placement by using semantic elements to define specific sections like navigation bars, sidebars, and main content areas. It is responsible for building the Registration Forms, where input types (text, email, password) are defined to ensure data is collected correctly. For the Video Call Layout, HTML provides the specific containers (video tags) that serve as the "windows" where the local and remote video streams are eventually rendered.

CSS (Cascading Style Sheets)

CSS transforms the raw HTML structure into a polished, user-friendly interface. It handles the visual identity of the app, including color palettes, typography, and spacing (padding/margins). A critical role of CSS in this project is Responsive Design; by using Media Queries and flexible layouts (like Flexbox or CSS Grid), the application ensures that the dashboard and video call interface remain functional and readable whether the user is on a wide desktop monitor or a narrow smartphone screen. It also provides visual feedback, such as changing button colors when hovered over, to improve the overall User Experience (UX).

JavaScript

JavaScript serves as the "brain" of the front end, enabling complex, logic-based interactions. Its most vital role here is managing WebRTC (Web Real-Time Communication), which involves a sophisticated handshake process to connect two users' cameras and microphones across the internet.

Furthermore, JavaScript handles DOM (Document Object Model) Manipulation. This allows the app to take translated text received from the server and inject it directly into the HTML page. Because this happens asynchronously, the user sees the translated captions appear in real-time during a call without the entire page having to refresh, which is essential for maintaining a continuous video stream.

Bootstrap

Bootstrap was integrated as the primary front-end framework to ensure a professional, consistent user interface. By utilizing its library of pre-designed components, the development process was significantly accelerated without extensive custom CSS. Its mobile-first philosophy ensures the parking marketplace dashboard and video call interface remain functional and visually sharp across all devices, from desktop monitors to smartphones.

Beyond aesthetics, Bootstrap's 12-column grid system was instrumental in organizing the layout, allowing video windows and MySQL data tables to align precisely. The framework also handles cross-browser compatibility, ensuring that interactive elements render reliably on Chrome, Firefox, and Safari. This provides a seamless, intuitive experience for users while maintaining structural integrity across the entire platform.

PyCharm

For the development phase, PyCharm was utilized as the Integrated Development Environment (IDE) to manage both the Python logic and the front-end components. Developed by JetBrains, PyCharm is widely regarded as the "gold standard" for Python development, providing a robust suite of tools like intelligent code completion, on-the-fly error checking, and a powerful graphical debugger.

Its built-in support for SQL databases and web technologies made it an ideal "command center" for this project, allowing for seamless transitions between writing back-end scripts and refining HTML/CSS files. Essentially, it acted as a highly capable co-pilot, catching minor syntax errors before they became major headaches and ensuring the overall project structure remained organized and scalable.

8.2.BACK END:

Python

Python is the core programming language powering the entire application. It was chosen for its clean syntax and extensive library support, which are essential for handling the backend's diverse tasks. From processing video call metadata to managing the integration of translation services, Python's versatility allows for a seamless connection between the front-end user interface and the underlying data structures. Its efficiency in handling asynchronous tasks ensures that the application remains responsive even during high-traffic periods.

Django

Django is a high-level Python web framework used to accelerate the development process while maintaining high security standards. It acts as the primary orchestrator, managing critical components such as the user authentication system, URL routing, and database models. By following the "Don't Repeat Yourself" (DRY) principle, Django simplifies the interaction with the MySQL database through its Object-Relational Mapper (ORM), allowing for complex data operations to be performed using Python code rather than raw SQL.

SQLyog

SQLyog is the graphical user interface (GUI) tool utilized to manage and monitor the MySQL database effectively. It provides a visual environment for creating tables, defining relationships, and executing SQL queries to verify that data—such as user bookings and payment records—is being stored correctly. This tool was instrumental during the development phase for debugging database schemas and ensuring that the structural integrity of the relational data was maintained across all normalization forms.

SYSTEM TESTING

9.SYSTEM TESTING

Testing is very important in determining the reliability and efficiency of software, and hence it is very crucial stage in software development. Tests are conducted on the software to evaluate its performance at different levels.

Testing is vital to the success of the system. It is the penultimate step of software development. An elaborate testing of data is prepared and the system is using test data. While doing testing errors are noted and correction is made. The users are trained to operate the developed system. Both hardware and software are made to run the developed system successfully.

Testing is a process of executing a program with the intent of finding whether the software achieves the desired result. Tests are conducted to locate an undiscovered error. Different types of data are fed into the system and the end result is verified with the expected results. System testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is vital to the success of the system.

The various testing methods used for this application are:

- Unit testing
- Integration testing
- Validation testing
- Output testing

UNIT TESTING

Unit testing focuses verification efforts on the smallest unit of software design module. To check whether each module in this software works properly so that it gives desired outputs to the given inputs. All validations and conditions are tested in the module level in the unit test control parts are tested to ensure the information correctly flows into, and out of the program

unit under test. Boundary conditions are tested to ensure that modules operate at the boundaries.

INTEGRATION TESTING

Data can be lost across an interface, one module have an adverse effect on the other sub-functions, when combined may not produce the desired functions. Integrated testing is the systematic to uncover the errors within the interface. This testing is done by inputting necessary values and data in the sequential order. As data of one unit is needed for working of other. The need for integrated system is to find the overall system performance.

VALIDATION TESTING

Validation test is defined with a simple definition that validation succeeds when the system functions in a manner that can be reasonably accepted by the customer. Validation is done to see whether the corresponding entries made in the tables are correct. Proper validations are done in case of insertion and updating of tables. If any such arises, then proper error messages or warning, if any, has to be displayed.

In this application the validation testing is done to different fields especially in registration and sign up field. Where the validation done through python program. Main validation used is NULL value in the text boxes. And also done Mobile number , e-mail validation.

OUTPUT TESTING

After performing validation testing, the next step is output testing of the proposed system. Since the system cannot be useful if it does not produce the required output. Asking the user about the format in which the system is required tests the output displayed or generated by the system under consideration. While testing, errors are again uncovered and corrected by using

the above steps and correction are also noted for future use. The system has been verified and validated by running test data and live data.

USER ACCEPTANCE TESTING

User acceptance testing of the system is the key factor for the success of any system. As we have created any easy interface it is easy to use. Only tough portion is the information to be perfect and has only one kind of user that the administrator.

The system under consideration is tested for user acceptance by constantly keeping in touch with prospective system at a time of the development and making change whenever required. This is done with regard to the input screen design and output screen design.

SYSTEM IMPLEMENTATION & SYSTEM MAINTAINANCE

10.1.SYSTEM IMPLEMENTATION

The implementation includes all those activities that take place to convert from the old system to new. The old system consists of no personal portal for the airmen to track with officials only official public website from the proposed new system. A proper implementation is essential to provide a reliable system to meet the requirements of the customers. An improper implementation may affect the success of the android application.

There are several methods for handling the implementation and the consequent conversion from the old applications to the new application developed in this project.

The most secure methods for compare the old system and the new system is to run the old and new system in parallel. In this approach, a person may operate the old existing application and the new application. This method offers high reliability and security.

working version of the system can is implemented in two parts, the website and android application. The website is managed by the admin and the airmen.

The implementation plan includes host the website and put it into its operation. The implementation plan consists of the following steps:

- List all files required for implementation.
- Host the website and put it into its operation.
- Host the application in a smart phone and put it into its use.

The implementation plan should anticipate possible problems and must be able to deal with them. The usual problems may be missing documents; mixed data formats between current files and errors in data translation, missing data etc.

USER TRAINING

The implementation of the proposed system includes the training of system operators. Training the system operators includes not only instructions in how to use the equipment, but also in how to diagnose malfunctions and in what steps to take when they occur. So proper training should be provided to the system operators. No training is complete without familiarizing users with simple system maintenance activities. Since the proposed system is developed in a GUI, training will be comparatively easy than systems developed in a non-GUI. There are different types of training.

We can select off-site to give depth knowledge to the system operators.

Success of the system depends on the way in which it is operated and used. Therefore, the quality of training given to the operating person affects the successful implementation of the system. The training must ensure that the person can handle all the possible operations.

Training must also include data entry personnel. They must also be given training for the installation of new hardware, terminals, how to power the system, how to power it down, how to detect the malfunctions, how to solve the problems etc. the operators must also be provided with the knowledge of trouble shooting which involves the determination of the cause of the problem.

The proposed system requires trained personnel for operating the system. This will reduce the data entry errors considerably. It is preferable to provide the person with some kind of operating manuals that will explain all the details of the system.

For the purpose of training we have improved our user interface for a guiding style of use and we are providing an intuitive interface for users. Along with all the simplicity we are providing a help section for users of the application with a detailed description of how each module is working and feature wise specialties and benefits.

10.2.SYSTEM MAINTAINANCE

Maintenance of the software is one of major step in the development of the computer system. Software, which is developed by the engineer, should undergo maintenance process in a regular interval of time as time on new problem arises and it must be corrected accordingly. Maintenance and enhancement are a long-term process.

In this project, the maintenance is carried over by the staff. Since they are the key persons to develop this project they know clearly about the project and coding structured. So, they will change the coding whenever required. Regarding the project maintenance, the changes will occur then and there according to the conditions

Various types of maintenance that can be made are:

- **Corrective maintenance:** reactive modification (or repairs) of a software product performed after delivery to correct discovered problems. Included in this category is emergency maintenance, which is an unscheduled modification performed to temporarily keep a software product operational pending corrective maintenance.
- **Adaptive maintenance:** modification of a software product performed after delivery to keep a software product usable in a changed or changing environment. For example, the operating system might be upgraded and some changes to the software may be necessary.
- **Perfect maintenance:** modification of a software product after delivery to provide enhancements for users, improvement of program documentation, and recoding to improve software performance, maintainability, or other software attributes.

- **Preventive Maintenance:** modification of a software product after delivery to detect and correct latent faults in the software product before they become operational faults.

The staff in the concern takes part in each and every level of the project. So they don't need any training of the software. During the development process they sat and entered each and every entry to test the project. They themselves used this is an opportunity to take training is not needed for the users.

SAMPLE CODE

11.SAMPLE CODE

Login page

```
{% extends "index.html" %}
{% block body %}

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
<link
href="https://fonts.googleapis.com/css2?family=Montserrat:wght@300;400;500;
600;700&family=Playfair+Display:wght@400;600&display=swap"
rel="stylesheet">
<html>
<head>
<style>
    /* VARIABLES */
    :root {
        --bg-dark: #0a0a0a;
        --bg-gradient: linear-gradient(135deg, #000000 0%, #1f1f1f 50%, #434343
100%);
        --glass-bg: rgba(20, 20, 20, 0.7);
        --glass-border: rgba(255, 255, 255, 0.08);
        --text-main: #e0e0e0;
        --text-muted: #a0a0a0;
        --input-bg: rgba(0, 0, 0, 0.4);
        --input-focus: rgba(0, 0, 0, 0.6);
    }

    /* GLOBAL RESET */
```

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* NEW WRAPPER: Styles previously applied to 'body' are now here */
.login-page-wrapper {
  font-family: 'Montserrat', sans-serif;
  background: var(--bg-dark);
  color: var(--text-main);
  /* Ensure it covers the full viewport height within the block */
  min-height: 100vh;
  width: 100%;
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  overflow: hidden;
  perspective: 1000px;
  position: relative;

  /* --- SIZE INCREASE IMPLEMENTATION --- */
  /* Scale up the entire content block by 10% */
  **transform: scale(1.1);** /* Ensure transformation origin is centered */
  **transform-origin: center center;**
  /* Add some padding to prevent scaled content from touching edges,
     especially if the original content was close to the viewport limits */
  **padding: 5vh 0;** }

```

```

/* --- 3D VOXEL BACKGROUND ANIMATION --- */
.voxel-container {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: -1;
    overflow: hidden;
    background: var(--bg-gradient);
}

.voxel {
    position: absolute;
    width: 40px;
    height: 40px;
    transform-style: preserve-3d;
    animation: floatVoxel 10s infinite linear;
}

/* Constructing a CSS Cube */
.voxel div {
    position: absolute;
    width: 40px;
    height: 40px;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
}

/* Cube Faces */
.voxel .front { transform: translateZ(20px); }
.voxel .back  { transform: rotateY(180deg) translateZ(20px); }

```

```

.voxel .right { transform: rotateY(90deg) translateZ(20px); }
.voxel .left { transform: rotateY(-90deg) translateZ(20px); }
.voxel .top { transform: rotateX(90deg) translateZ(20px); }
.voxel .bottom { transform: rotateX(-90deg) translateZ(20px); }

@keyframes floatVoxel {
  0% { transform: translateY(110vh) rotateX(0deg) rotateY(0deg); opacity: 0; }
  20% { opacity: 1; }
  80% { opacity: 1; }
  100% { transform: translateY(-20vh) rotateX(360deg) rotateY(360deg);
opacity: 0; }
}

/* --- LOGIN CARD --- */
.login-wrapper {
  position: relative;
  width: 100%;
  max-width: 420px;
  padding: 20px;
  z-index: 10;
}

.login-card {
  background: var(--glass-bg);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  border: 1px solid var(--glass-border);
  border-radius: 20px;
  padding: 40px 30px;
  box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
  transform-style: preserve-3d;
  animation: cardFloat 6s ease-in-out infinite;

```

```

    position: relative;
}

.login-card::after {
    content: "";
    position: absolute;
    inset: -1px;
    z-index: -1;
    border-radius: 20px;
    background: linear-gradient(45deg, transparent, rgba(255,255,255,0.1),
transparent);
}

@keyframes cardFloat {
    0%, 100% { transform: translateY(0) rotateX(0); }
    50% { transform: translateY(-10px) rotateX(1deg); }
}

/* --- BRANDING --- */
.brand-section {
    text-align: center;
    margin-bottom: 30px;
}

.logo-icon {
    font-size: 48px;
    margin-bottom: 10px;
    background: linear-gradient(to right, #ffffff, #888888);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    display: inline-block;
    animation: pulseHand 3s infinite ease-in-out;

```

```

}

@keyframes pulseHand {
  0% { transform: scale(1); filter: drop-shadow(0 0 0px rgba(255,255,255,0)); }
  50% { transform: scale(1.05); filter: drop-shadow(0 0 10px
rgba(255,255,255,0.2)); }
  100% { transform: scale(1); filter: drop-shadow(0 0 0px
rgba(255,255,255,0)); }
}

.brand-section h1 {
  font-family: 'Montserrat', sans-serif;
  font-weight: 700;
  font-size: 28px;
  letter-spacing: 3px;
  text-transform: uppercase;
  color: #fff;
  text-shadow: 0 4px 10px rgba(0,0,0,0.5);
}

.brand-section p {
  font-size: 11px;
  color: var(--text-muted);
  letter-spacing: 2px;
  margin-top: 5px;
  text-transform: uppercase;
}

/* --- FORM INPUTS --- */
.input-group {
  position: relative;
  margin-bottom: 25px;

```

```
width: 100%;
}

.input-group i.fa-user,
.input-group i.fa-lock {
    position: absolute;
    left: 15px;
    top: 50%;
    transform: translateY(-50%);
    color: #666;
    transition: 0.3s;
    z-index: 2;
}

/* Toggle Icon Style */
.input-group .password-toggle-icon {
    position: absolute;
    right: 15px;
    top: 50%;
    transform: translateY(-50%);
    color: #666;
    cursor: pointer;
    z-index: 2;
    transition: color 0.3s;
}

.input-group .password-toggle-icon:hover {
    color: #e0e0e0;
}

.input-field {
    width: 100%;
```

```
padding: 14px 45px 14px 45px; /* Padding for both icons */
background: var(--input-bg);
border: 1px solid #333;
border-radius: 10px;
color: white;
font-size: 14px;
font-family: 'Montserrat', sans-serif;
transition: all 0.3s ease;
}
```

```
.input-field:focus {
  outline: none;
  border-color: #888;
  background: var(--input-focus);
  box-shadow: 0 0 15px rgba(255, 255, 255, 0.05);
}
```

```
.input-field:focus + i {
  color: #fff;
}
```

```
/* --- BUTTONS & LINKS --- */
.submit-btn {
  width: 100%;
  padding: 14px;
  border: none;
  border-radius: 10px;
  background: #e0e0e0;
  color: #000;
  font-family: 'Montserrat', sans-serif;
  font-weight: 700;
  font-size: 14px;
}
```

```
text-transform: uppercase;
letter-spacing: 1px;
cursor: pointer;
transition: all 0.3s ease;
margin-top: 10px;
box-shadow: 0 5px 15px rgba(0,0,0,0.3);
}

.submit-btn:hover {
  background: #fff;
  transform: translateY(-2px);
  box-shadow: 0 8px 20px rgba(255,255,255,0.1);
}

.links-container {
  margin-top: 25px;
  text-align: center;
  font-size: 13px;
  color: var(--text-muted);
}

.links-container a {
  color: #fff;
  text-decoration: none;
  font-weight: 500;
  position: relative;
  transition: 0.3s;
}

.links-container a:hover {
  text-decoration: underline;
}
```

```

.forgot-pass {
    display: block;
    margin-top: 15px;
    font-size: 12px;
    opacity: 0.7;
}

.forgot-pass:hover {
    opacity: 1;
}

/* --- MESSAGE BOX --- */
.alert-box {
    padding: 12px;
    border-radius: 8px;
    font-size: 13px;
    text-align: center;
    margin-bottom: 20px;
    background: rgba(255,255,255,0.1);
    border: 1px solid rgba(255,255,255,0.2);
}
</style>
</head>
<body>
<div class="login-page-wrapper">

    <div class="voxel-container" id="voxelContainer">
        </div>

    <div class="login-wrapper">
        <div class="login-card">

```

```

<div class="brand-section">
  <div class="logo-icon">
    <i class="fas fa-hands-asl-interpreting"></i>
  </div>
  <h1>SIGN LINK</h1>
  <p>Bridging the Gap</p>
</div>

```

```

{% if message %}
<div class="alert-box">
  {{ message }}
</div>
{% endif %}

```

```

<form id="form1" name="form1" method="post" action="/login_button">

```

```

  <div class="input-group">
    <i class="fas fa-user"></i>
    <input type="email" id="email" name="username" class="input-field"
placeholder="Email Address"
      pattern="^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
required
      oninvalid="this.setCustomValidity('Please enter a valid email
(e.g., user@example.com)')"
      oninput="this.setCustomValidity('')">
  </div>

```

```

  <div class="input-group">
    <i class="fas fa-lock"></i>
    <input type="password" id="psw" name="passwd" class="input-
field" placeholder="Password"

```

```

        pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}" required
        oninvalid="this.setCustomValidity('Password must contain at
least one number, one uppercase and lowercase letter, and be at least 8
characters long')"
        oninput="this.setCustomValidity('')">
        <i class="fas fa-eye password-toggle-icon" id="togglePassword"></i>
    </div>

    <input type="submit" name="button" id="button" value="Login"
class="submit-btn" />

    <div class="links-container">
        <p>New to Sign Link? <a href="/User/register">Create
Account</a></p>
        <a href="/forgotemail" class="forgot-pass">Forgot Password?</a>
    </div>

    </form>
</div>
</div>

<script>
document.addEventListener('DOMContentLoaded', () => {
    // --- Voxel Generation ---
    const container = document.getElementById('voxelContainer');
    const voxelCount = 20;

    if (container) {
        for (let i = 0; i < voxelCount; i++) {
            createVoxel(i);
        }
    }
}

```

```

function createVoxel(i) {
    const voxel = document.createElement('div');
    voxel.classList.add('voxel');

    // Add 6 faces for the cube
    const faces = ['front', 'back', 'right', 'left', 'top', 'bottom'];
    for (let j = 0; j < faces.length; j++) {
        const div = document.createElement('div');
        div.classList.add(faces[j]);
        voxel.appendChild(div);
    }

    // Random positioning and timing
    const left = Math.random() * 100;
    const delay = Math.random() * 10;
    const duration = 10 + Math.random() * 15;
    const scale = 0.5 + Math.random() * 0.8;

    voxel.style.left = left + '%';
    voxel.style.animationDelay = '-' + delay + 's';
    voxel.style.animationDuration = duration + 's';
    voxel.style.transform = 'scale(' + scale + ')';

    container.appendChild(voxel);
}

// --- Show/Hide Password ---
const togglePassword = document.getElementById('togglePassword');
const password = document.getElementById('psw');

if(togglePassword && password) {

```

```

togglePassword.addEventListener('click', function () {
    // toggle the type attribute
    const type = password.getAttribute('type') === 'password' ? 'text' :
'password';
    password.setAttribute('type', type);

    // toggle the eye / eye slash icon
    if(this.classList.contains('fa-eye')) {
        this.classList.remove('fa-eye');
        this.classList.add('fa-eye-slash');
    } else {
        this.classList.remove('fa-eye-slash');
        this.classList.add('fa-eye');
    }
});
}
});
</script>

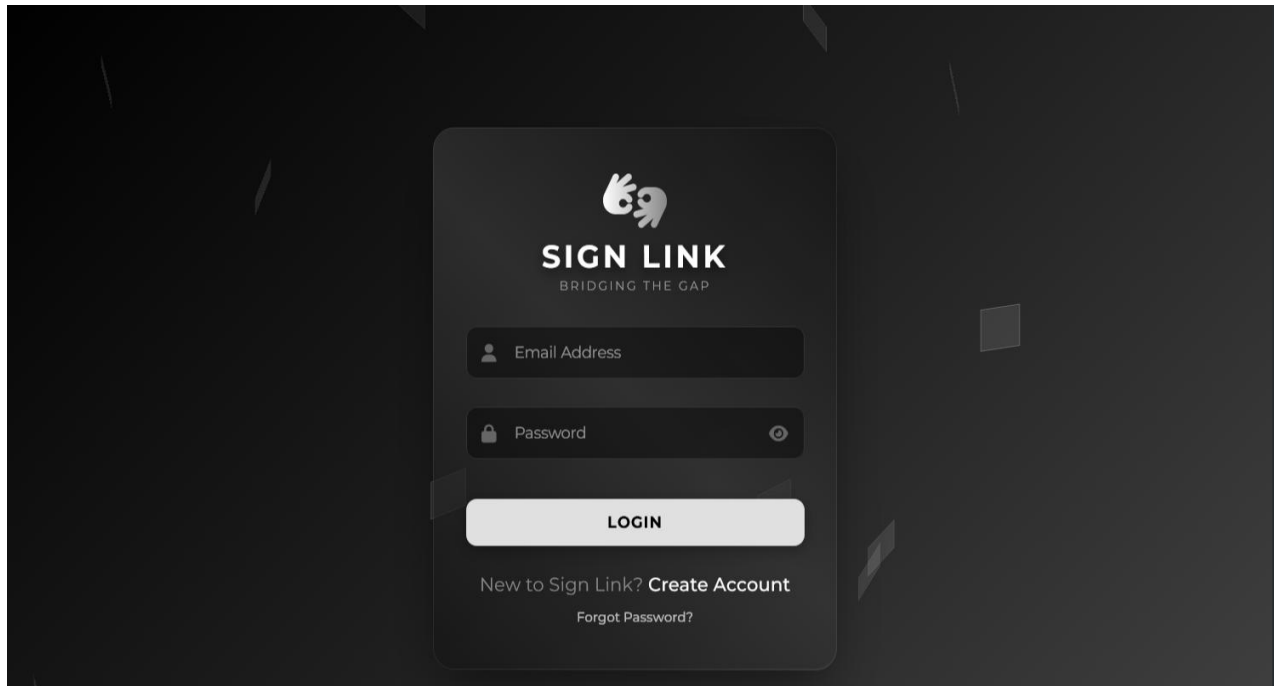
</div>
</body>
</html>
{% endblock %}

```

SCREENSHOT

12.SCREENSHOT

LOGIN PAGE



The screenshot displays a login interface for 'SIGN LINK' with the tagline 'BRIDGING THE GAP'. The interface is centered on a dark background. It features a logo at the top, followed by two input fields for 'Email Address' and 'Password'. A prominent 'LOGIN' button is positioned below the password field. At the bottom, there are links for 'Create Account' and 'Forgot Password?'. The entire form is enclosed in a dark, rounded rectangle.

SIGN LINK
BRIDGING THE GAP

Email Address

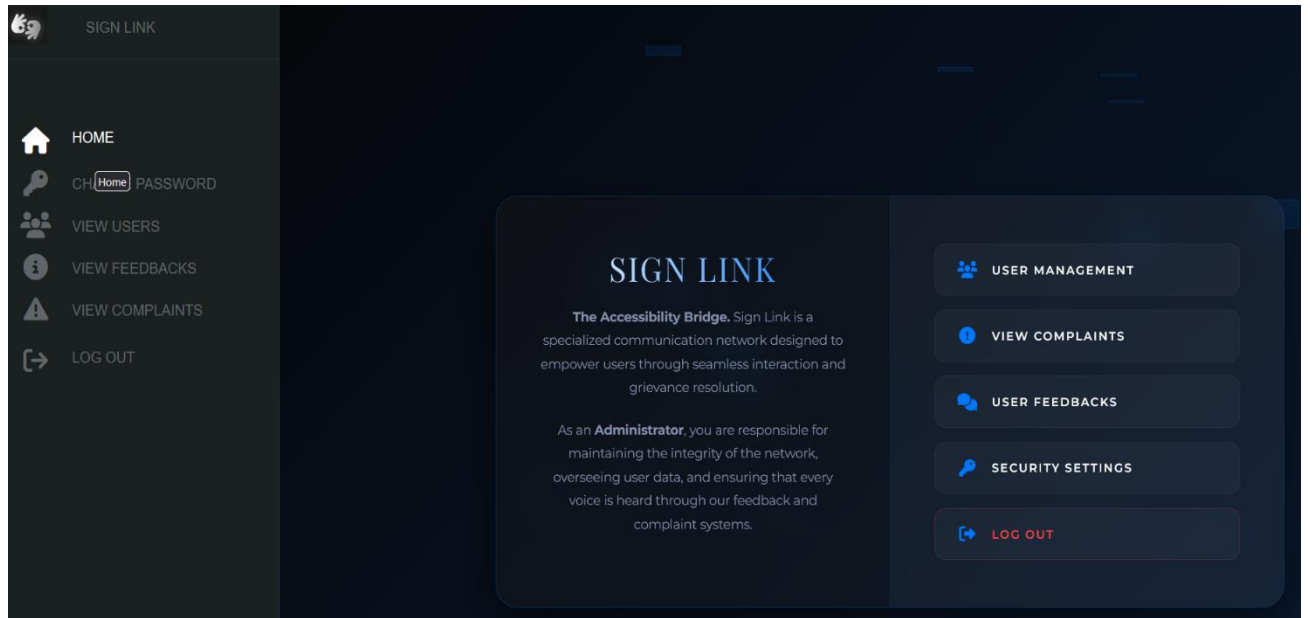
Password

LOGIN

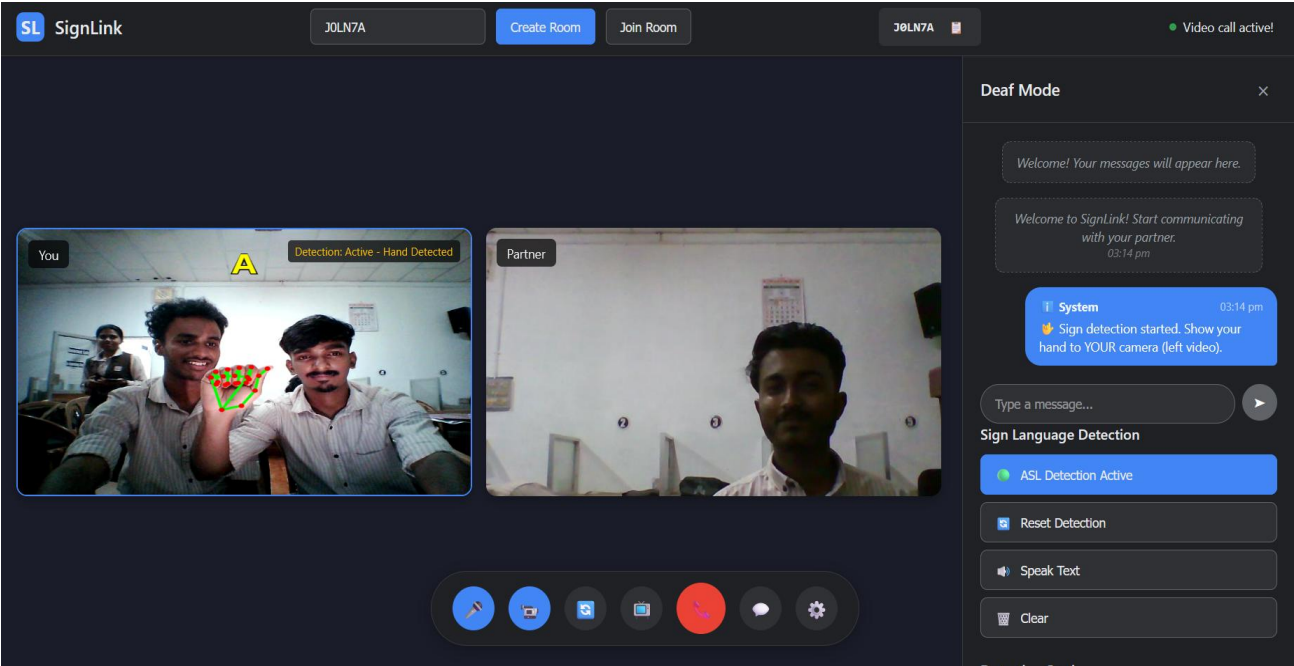
New to Sign Link? [Create Account](#)

[Forgot Password?](#)

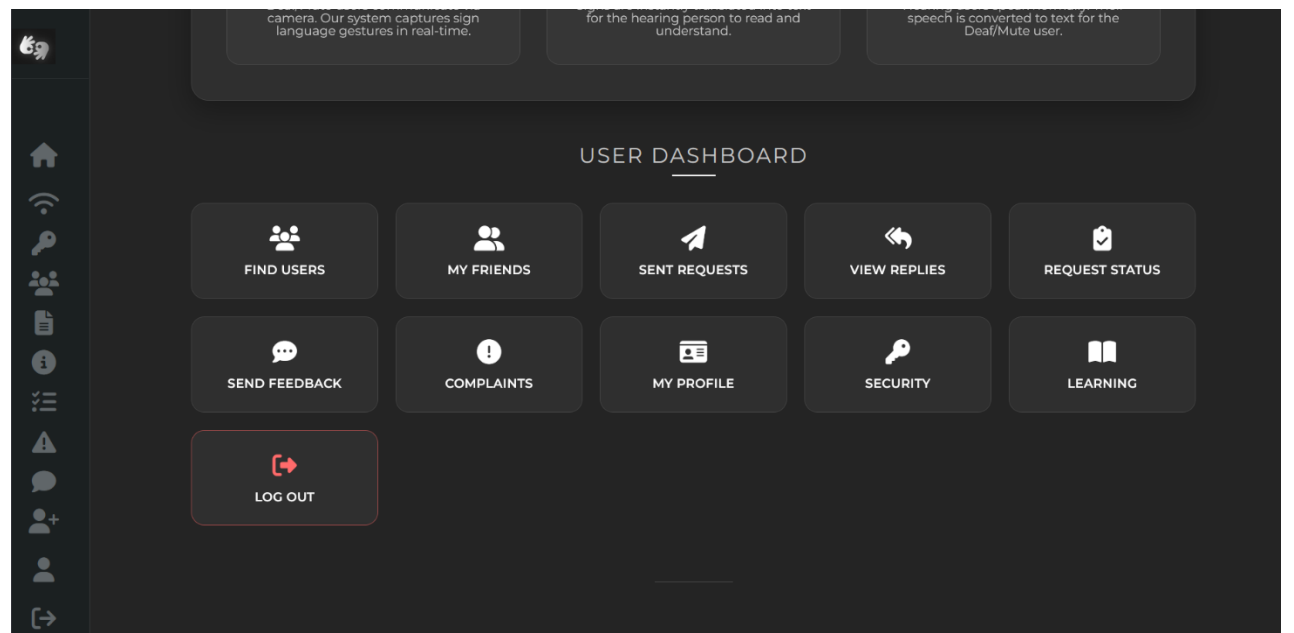
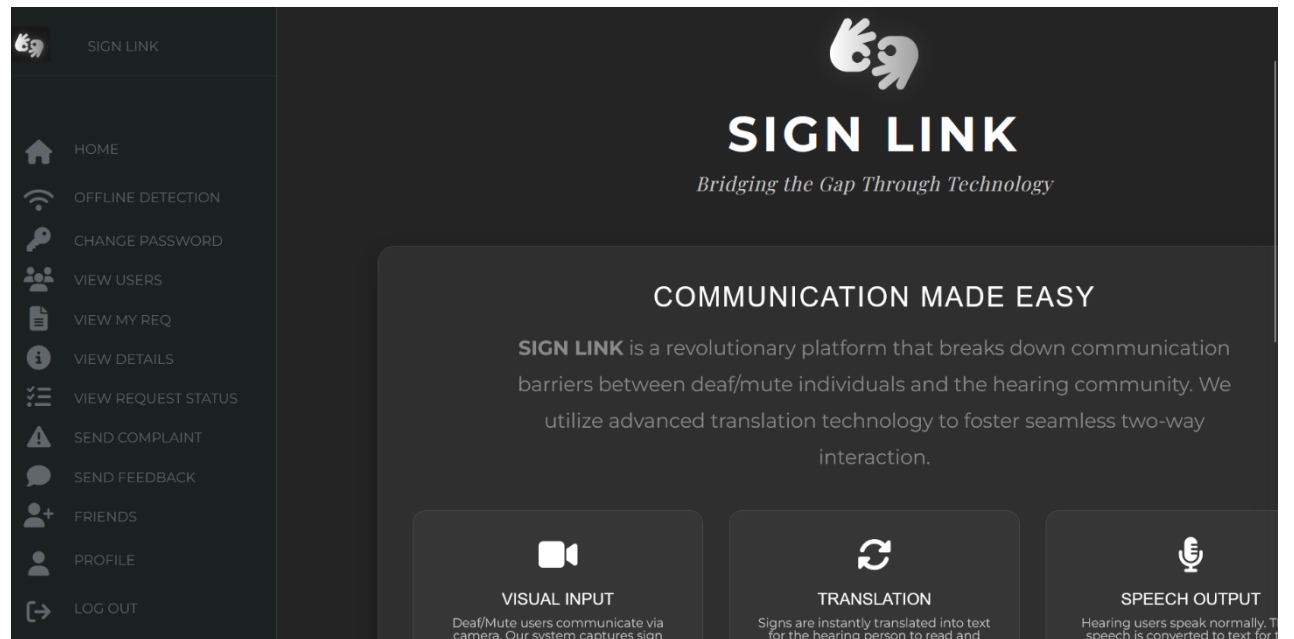
ADMIN HOME PAGE



VIDEO CALL PAGE



USERS HOME PAGE



SCOPE FOR FUTURE ENHANCEMENT

13.SCOPE FOR FUTURE ENHANCEMENT

Scope of the Project

The proposed system is designed to bridge the communication gap between the deaf and hearing communities by providing a seamless, browser-based platform that translates sign language into text in real-time. By utilizing WebRTC for low-latency video streaming and Deep Learning for gesture recognition, the system allows for fluid conversations using standard webcams rather than specialized hardware, making it highly accessible for both social and professional settings. This scope is supported by a robust Django and MySQL back end, which ensures secure user management and data integrity, while a Bootstrap-powered front end provides a responsive experience across desktops and tablets. Ultimately, the system provides an end-to-end solution that combines sophisticated AI translation with a structured, scalable web architecture to facilitate inclusive communication.

1. Mobile Application

While the current web-based platform is functional, converting the system into a dedicated mobile application using frameworks like Flutter or React Native would significantly increase user accessibility. A mobile-first approach allows users to communicate on the go, leveraging native hardware optimizations for camera stability and push notifications for incoming call alerts. This transition ensures that the service is not confined to a desktop environment, making it a truly portable solution for real-time translation in everyday scenarios.

2. Multi-Language Support

To cater to a broader demographic, the system's translation capabilities will be expanded beyond English to include regional languages such as Malayalam and Hindi. By integrating more localized Natural Language Processing (NLP) models, the application can bridge communication gaps for users who are more comfortable in their native tongues. This expansion is crucial for making the technology inclusive and useful within diverse linguistic communities where English may not be the primary language of interaction.

3. Expanded Sign Library

The current recognition system is being scaled from basic alphabets and isolated words to a much larger dataset capable of interpreting complex sentences and grammatical nuances. By utilizing advanced deep learning architectures, the system will be trained to recognize the fluid movements and facial expressions that characterize natural sign language. This transition from "fingerspelling" to "continuous sign language recognition" is essential for achieving a more natural and conversational flow between users.

4. Voice Synthesis (Text-to-Speech)

To create a truly bidirectional communication experience, a Voice Synthesis feature will be integrated to convert the deaf user's translated text into artificial speech. This allows the hearing user to listen to the communication through their speakers or headphones, making the interaction feel more like a standard voice call. By using high-quality Text-to-Speech (TTS) engines, the system can provide various vocal tones and speeds, ensuring the translated message is delivered clearly and effectively to the hearing participant.

CONCLUSION

14.CONCLUSION

The Sign Link project successfully demonstrates the transformative power of technology in addressing significant social challenges. By seamlessly integrating modern web development with Machine Learning and Real-Time Communication (WebRTC), we have developed a platform that empowers deaf and hard-of-hearing individuals to communicate independently. This project moves beyond a simple technical exercise, serving as a vital bridge that fosters inclusion and accessibility in an increasingly digital world.

The application effectively dismantles traditional communication barriers, allowing individuals to "speak" through hand gestures that are instantly converted into text for the recipient. By leveraging Python, Django, and JavaScript, the system provides a stable, high-performance environment capable of handling complex data processing without sacrificing user experience. The inclusion of a MySQL database ensures that user data and system configurations remain secure and organized, proving that sophisticated accessibility tools can be built efficiently using standardized web technologies.

While the current implementation covers fundamental communication needs, it establishes a robust and scalable foundation for future innovation. This project highlights the potential for AI-driven tools to become a standard feature in global communication platforms, ensuring that no individual is left behind due to a physical disability. Ultimately, this work stands as a testament to how human-centric design, when paired with technical precision, can create a more equitable and connected society.

REFERENCES

15.REFERENCES

REFERENCE BOOK

1. Ian Somerville -"Software Engineering" 7thEdition, 2004
2. S Roger Presman, —Software Engineering|McGraw-hill|InternationalEdition, 1994
3. Alen dennis - "System Analysis and Design" 10th Edition, 2015
4. Rohith Khurana —Software Engineering principles and practices|2nd Edition, 2008

WEB REFERENCES

1. <http://www.stackoverflow.com>, StackOverflow
2. <http://www.codeproject.com>, CodeProject
3. <http://w3schools.com>, *W3School*